Proactive Autonomous Navigation System for Air Purifiers

Haewon Kwon
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
hw030302@hanyang.ac.kr

Dongjin Son
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
djson322@hanyang.ac.kr

Yena Jo
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
yena1743@hanyang.ac.kr

A. ***Abstract— This project proposes an AI-Based Proactive Autonomous Mobile Air Purification System for multiroom residential environments.***

The system predicts room-level air quality using machine learning and autonomously selects purification destinations based on predicted pollution and travel distance. It uses the A* algorithm for navigation and provides real-time monitoring and control through a mobile application. The proposed system will be implemented and evaluated in a software-based simulation to examine the effectiveness of proactive air purification.

Keywords— Air Quality Prediction, Machine Learning, Autonomous Navigation, Proactive Purification, Simulation

Roles	***Name***	***Task description and etc.***
User	*Yena Jo*	*Evaluates the mobile application's usability, tests monitoring and control functions, and provides feedback from the user's perspective.*
Customer	*Yena Jo*	*Defines project requirements, reviews system functionality, and evaluates whether the final results meet the project objectives.*
Software Developer	*Haewon Kwon*	*Develops the mobile application, backend, AI prediction model, navigation algorithm, and simulation, and performs system testing.*
Development Manager	*Dongjin Son*	*Manages project schedules, assigns tasks, coordinates team collaboration through GitHub, and oversees documentation and deliverables.*

II. INTRODUCTION

A. *Motivation*

1) Indoor air quality is an important factor in maintaining a comfortable residential environment and continuously changes over time and across different locations. In residential environments consisting of multiple rooms, such as living rooms, kitchens, and bedrooms, the timing and severity of air pollution may vary depending on the purpose of each space and the activities performed within it. For example, air quality in the kitchen may deteriorate during cooking hours, while different air quality patterns may occur in living rooms and bedrooms due to daily activities such as cleaning. Therefore, effective indoor air quality management requires consideration not only of conditions at a specific location but also of room-level air quality changes associated with recurring household activity patterns.

2) Conventional stationary air purifiers primarily purify the air around their installed locations and require users to manually relocate them to purify the air in other rooms. This approach has limitations in responding flexibly to changing air quality conditions in multiroom residential environments. For example, if air quality deteriorates in the kitchen while an air purifier is located in the living room, the purifier cannot directly perform purification in the kitchen unless the user moves it. Furthermore, an operating strategy based solely on currently measured air quality cannot proactively identify and respond to rooms where pollution is likely to occur in the near future. These limitations motivated the development of an air purification system capable of responding autonomously to room-level air quality changes while reducing the need for repeated manual relocation and user intervention.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

B. *Problem Statement (Client's Needs)*

1) Physical Limitations of Stationary Air Purifiers in Multiroom Environments

Conventional stationary air purifiers operate primarily within the space where they are installed. To manage air quality across multiple rooms, users must either manually relocate the purifier or install separate purifiers in individual rooms. However, installing an air purifier in every room may impose additional costs and space requirements, while manually moving a single purifier requires continuous user involvement.

Furthermore, air quality may deteriorate at different times in different rooms. Keeping an air purifier in the same location therefore limits its ability to respond flexibly to room-specific purification needs. Accordingly, a system is needed that can autonomously identify and move to rooms requiring purification without direct user intervention.

2) Limitations of Reactive Air Purification Based on Current Air Quality

Indoor air quality may exhibit recurring patterns associated with household activities such as cooking, cleaning, and ventilation, as well as time-dependent environmental changes. However, when purification is initiated solely on the basis of currently measured air quality, the system may respond only after air quality has already deteriorated.

For example, even if air quality in the kitchen repeatedly deteriorates at a particular time of day, an air purifier that does not utilize such patterns may begin responding only after pollution has occurred. Therefore, a proactive air quality management approach is needed to learn from historical data, predict future air quality in individual rooms, and determine appropriate purification locations and timing before pollution becomes severe.

3) *Unnecessary Movement and Inefficient Purification Destination Selection*

If a mobile air purifier repeatedly visits every room in a predetermined sequence, it may travel unnecessarily regardless of actual purification needs. Conversely, selecting only the room with the poorest current air quality may fail to account for rooms where air quality is expected to deteriorate in the near future.

When multiple rooms require purification, the system must consider not only the anticipated pollution risk but also the purifier's current location and the travel distance to each destination. Therefore, an intelligent decision-making mechanism is needed to determine purification priorities by considering future air quality predictions and movement costs, while planning efficient navigation routes.

4) Need for Autonomous Decision-Making and Replanning in Dynamic Environments

Future air quality predictions are based on historical data and may not always accurately reflect actual environmental changes. When the purifier arrives at its destination, the measured air quality may differ from the predicted conditions. In addition, obstacles or changes in available routes may prevent the purifier from executing its original movement plan.

If the system continues to follow its initially selected destination and route without modification, it cannot respond flexibly to such changes. Therefore, the purifier must be able to reassess the need for its current task using new observations from its onboard sensor and, when necessary, modify its destination, navigation route, or purification plan.

5) User Burden in Air Purifier Management and the Need for Automatic Control

Managing air quality across multiple rooms can be inconvenient when users must continuously monitor room conditions and repeatedly determine where the air purifier should operate. In environments where air quality changes over time, it may also be difficult for users to identify the most appropriate purification location and timing.

Therefore, a system is needed that automatically performs navigation and purification based on historical data and newly observed information while allowing users to easily monitor the purifier's current location, predicted air quality, and automatic control status. Users should also be able to interrupt automatic operation and manually adjust the purifier's behavior when necessary.

C. *Solution*

To overcome the spatial limitations of conventional stationary air purifiers and improve reactive air purification strategies, this project proposes an AI-Based Proactive Autonomous Mobile Air Purification System. The proposed system learns from historical air quality data to predict future air quality in individual rooms and autonomously moves to locations where purification is expected to be needed. By doing so, it aims to reduce the burden on users to manually relocate the purifier or determine when purification should occur, while enabling more proactive air quality management across multiple rooms.

1) AI-Based Future Air Quality Prediction and Proactive Purification

The system analyzes historical air quality data and time-dependent patterns to predict future air quality in individual rooms. Based on these predictions, it identifies rooms where air quality is expected to deteriorate and enables purification to begin before pollution becomes severe.

2) Autonomous Navigation and Adaptive Control

The system autonomously selects and navigates to a target room by considering predicted air quality and travel distance. Upon arrival, it measures the surrounding air quality using its onboard sensor and reassesses its movement and purification plans when new measurements or environmental changes occur. Rather than repeatedly following a predetermined route, the system provides autonomous air quality management by adapting its actions to changing conditions.

3) Real-Time Monitoring and User Control

Users can monitor the purifier's current location, movement status, room-level air quality predictions, and purification status through a mobile application. The system also provides automatic and manual control modes, allowing users to enable autonomous operation or directly control the purifier when necessary.

Our team aims to implement these functions in a software based virtual residential environment and evaluate the effectiveness of proactive air purification and autonomous navigation by comparing the proposed approach with a fixed movement strategy.

D. Research on Any Related Software

1) LG PuriCare AI Objet Collection 360° Air Purifier M7

a. Function

The LG PuriCare AI Objet Collection 360° Air Purifier M7 monitors indoor air quality and automatically adjusts purification intensity. Its AI Custom Operation function learns indoor air-quality patterns, predicts pollution changes, and optimizes purification accordingly [1], [2], [3].

b. Simlarity with our Service

Both systems use AI-based air-quality prediction to improve purification efficiency. They aim to respond to anticipated pollution rather than relying solely on current air-quality measurements.

c. Difference from our Service

LG M7 uses predicted air-quality changes to adjust purification intensity at its installed location. In contrast, our system predicts room-level PM2.5 concentrations 30 minutes ahead and autonomously moves to the room requiring purification, considering both predicted pollution risk and travel cost.

2) ECOVACS AIRBOT Z1

a. Function

ECOVACS AIRBOT Z1 is an autonomous mobile air-purification robot that maps indoor spaces and navigates to designated areas. Its Linked Purification function allows the robot to automatically move to a specific area and begin purification when a connected air-quality monitor detects pollution above a configured threshold [4].

b. Simlarity with our Service

Both systems combine autonomous navigation and air purification. They can move between indoor areas and perform purification based on air-quality information.

c. Difference from our Service

ECOVACS AIRBOT Z1 initiates movement when a connected monitor detects that air pollution has reached a predefined threshold. In contrast, our system uses AI to predict PM2.5 concentrations 30 minutes ahead and proactively selects a destination before the anticipated pollution increase. It also considers travel cost when determining the destination.

III. REQUIREMENTS

A. Application Launch and User Login

1) Initial access:
When the mobile application is opened, a short loading screen shall be displayed while the application checks whether a valid login session exists. A user without a valid session shall be directed to the login screen; an authenticated user shall proceed to the main dashboard. The application is intended for a local demonstration and does not require actual LG appliance registration.

2) Login credentials and validation:
The login screen shall provide an email field, a password field with masked input, and a Sign In button. Empty fields and incorrectly formatted email addresses shall be rejected before a request is sent. For the prototype, a demonstration

account will be prepared in advance rather than implementing a public sign-up service

3) Authentication result and logout:
When valid credentials are submitted, the system shall authenticate the user and open the dashboard. Incorrect credentials shall produce an error message without revealing whether an account exists. A logout option shall end the current session and return the user to the login screen. If the server is unavailable, the user shall see a connection error instead of a misleading success message.

B. Main Dashboard and Simulation Control

1) Main screen:
After login, the application shall display a virtual apartment floor plan, the air purifier's location, the simulation clock, and the current system mode. The apartment shall initially consist of a living room, kitchen, and bedroom connected by valid passages. The map shall show walls and fixed furniture as obstacles.

2) Start, pause, and reset:
A user shall be able to start, pause, resume, and reset the virtual simulation. Starting a run shall begin the virtual clock and environmental updates. Pausing shall freeze both environmental changes and purifier movement without deleting the current run. Resetting shall restore the initial position, time, and air quality conditions. Repeated or incompatible commands shall not create duplicate simulation processes.

3) Mode selection:
The user shall be able to choose Automatic or Manual mode. In Automatic mode, the controller shall select movement destinations. In Manual mode, the user shall select a room to visit and shall be able to turn purification on or off. A change of mode shall stop any conflicting movement command.

C. Virtual Indoor Air Quality and Measurement

1) Air quality changes over time:
The system shall simulate PM2.5 concentration in each room rather than using fixed values. Events such as kitchen cooking, living-room cleaning, and bedroom dust disturbance shall affect concentration with different probabilities, durations, and intensities. Natural dissipation and purification shall reduce pollution. The same daily pattern shall not be repeated without variation.

2) Historical data preparation:
For initial model development, the system shall be capable of generating 60 virtual days of room-level PM2.5 data at 10-minute intervals. With three rooms, this represents approximately 25,920 room-time records. These figures describe the planned synthetic dataset, not measurements collected from real households.

3) Measurement constraints:
The virtual purifier shall obtain a new measurement only in the room where it is physically located in the simulation. For rooms it has not visited recently, the system shall retain the last measurement together with its timestamp. The application shall distinguish a current measurement, an old measurement, and an AI prediction. The autonomous controller shall not receive the simulator's hidden real-time values for unvisited rooms.

D. Future Air Quality Prediction

1) Prediction request and outcome:
The system shall estimate the PM2.5 concentration of each room 30 virtual minutes ahead using a machine-learning model. Predictions shall be updated periodically while the simulation is running in Automatic mode, and the user shall be able to inspect the forecast for each room.

2) Available input information:
Forecasting shall use information available at decision time, such as room identity, hour, day of the week, last observed PM2.5, time elapsed since the last observation, and historical patterns derived from previous measurements. The model shall not use pollution events or measured concentrations from the future as input.

3) Prediction status:
The application shall identify predicted values as estimates rather than direct measurements. If the trained model cannot be loaded or insufficient historical observations are available, the controller shall show that prediction is unavailable and switch to a clearly identified non-predictive fallback mode instead of fabricating a result.

E. Autonomous Movement and Proactive Purification

1) Destination selection:
In Automatic mode, the controller shall use future air quality estimates to decide whether a room is likely to need attention. It shall consider both predicted pollution and travel distance, rather than always choosing the room with the highest predicted value. The purifier may depart for a room before its pollution rises to a severe level.

2) Route and travel:
After selecting a destination, the system shall calculate a route through walkable cells and doorways. The purifier shall move along this route step by step and shall not pass through walls or fixed obstacles. If no valid route exists, the system shall remain in a safe virtual position and record the failure.

3) Purification after arrival:
On arrival, the purifier shall perform a local measurement and begin purification when the current condition or

predicted near-future risk warrants it. The simulation shall represent a gradual reduction in PM2.5 rather than an immediate change to a clean value. A new measurement or a changed forecast may trigger reconsideration of the operating plan.

4) Unexpected conditions:
If the purifier discovers unexpectedly high pollution in its current room, the system shall give that observed condition priority over a routine forecast-based trip. To avoid continuous switching between rooms, routine destination changes shall use a minimum operating interval unless an urgent measured condition is detected.

F. Monitoring, History, and User Feedback

1) Live visualization:
The dashboard shall show the purifier moving on the floor plan, the current room, movement destination, and operating status (Idle, Moving, Measuring, or Purifying). The displayed air-quality values shall include their source and reference time so that predicted values are not confused with measured values.

2) Decision explanation and history:
The application shall present a short reason for an automatic decision, such as anticipated kitchen pollution or a newly measured high concentration. Users shall be able to review past predictions, visited rooms, purification activities, and significant status changes for the current or saved simulation run.

3) Communication failure:
If the mobile app loses its connection to the local server, it shall stop presenting old information as a live state. Reconnecting shall retrieve the current simulation state without restarting the environment or sending duplicate movement commands.

G. Evaluation of the Proactive Strategy

1) Comparison conditions:
The team shall compare the predictive controller with a non-predictive controller that visits rooms in a fixed sequence. Both methods shall be tested using the same apartment layout, environmental scenarios, initial conditions, sensing rules, purifier performance, and movement speed.

2) Performance measures:
The comparison shall report average room PM2.5 concentration, simulated time spent above a selected experimental pollution threshold, and total movement distance. Forecasting accuracy shall be evaluated separately on previously unseen synthetic scenarios. The evaluation shall not claim proven performance in real households.

IV. DEVELOPMENT ENVIRONMENT

A. Choice of Software Development Platform

1. Development Platform

(a) The project will be developed on macOS using Apple MacBook Air computers owned by team members. The operating systems are macOS Ventura, Sequoia, and Tahoe, with Apple M2–M3 processors and 8–16 GB of RAM.

(b) The system consists of a React Native mobile application and a Python backend. As the prototype is implemented through software simulation rather than construction of physical air-purifier hardware, existing development equipment can be used without additional hardware purchases.

(c) Initial development and testing will be conducted locally. AWS or Google Cloud may be considered later if cloud deployment becomes necessary.

2. Programming Languages

1) Python

(a) Python will be the main backend language, used for training AI models, generating and preprocessing data, planning autonomous routes, and developing the virtual-environment simulation.

(b) Python is widely used in machine learning and data analysis and supports libraries such as scikit-learn, Pandas, and NumPy. These tools simplify AI model development and data processing, while FastAPI enables communication with the mobile application.

(c) Because this project focuses on software-based AI prediction and simulation rather than physical hardware, Python was selected for its mature libraries and extensive development resources.

2) TypeScript

(a) TypeScript will be used to develop the React Native mobile application, including the interface through which users monitor and control the system.

(b) As a language that adds static typing to JavaScript, TypeScript can help detect type-related errors during development and improve code stability and maintainability.

(c) React Native also enables iOS and Android applications to be developed from a shared codebase, supporting efficient implementation within the limited project schedule.

3. Development Tools and Technologies

1) React Native + Expo — Mobile Application Development

(a) React Native is a cross-platform framework for developing iOS and Android applications using JavaScript or TypeScript. It will be used to implement the mobile dashboard and user controls.

(b) Expo supports React Native project setup, execution, and testing. By reducing the complexity of native development configuration and facilitating testing on mobile devices, it is suitable for iterative prototype development.

(c) Together, these tools support a shared codebase and reduce development and maintenance effort.

2) FastAPI — Backend Development

(a) FastAPI is a Python web API framework used to exchange data between the mobile application and the backend.

(b) Because AI prediction, simulation, and pathfinding will also be implemented in Python, FastAPI allows these components to be integrated within the same language ecosystem.

(c) It supports REST APIs, WebSocket connections, and automatic API documentation, which facilitate mobile–backend integration and testing.

3) scikit-learn — Machine Learning

(a) scikit-learn is a Python machine-learning library offering supervised-learning algorithms and evaluation utilities. The project will use it to implement and train a Random Forest regression model.

(b) Its preprocessing, training, prediction, and evaluation tools remove the need to implement a learning algorithm from scratch.

(c) Random Forest can learn nonlinear relationships among inputs without the computational demands of a large neural network, making it a practical choice for this synthetic-data prototype.

4) Pandas + NumPy — Data Processing and Analysis

(a) Pandas and NumPy are Python libraries for data handling and numerical computation. They will support generation and management of data used for model training and simulation.

(b) Pandas supports tabular data, time-series processing, missing-value handling, and preprocessing. NumPy provides array-based calculations and random-number generation for simulation data.

(c) Their compatibility with scikit-learn allows data generation, model training, and result analysis to remain within a consistent Python workflow.

5) A Algorithm — Autonomous Pathfinding*

(a) A* is a search algorithm for finding a route from a start location to a goal. It will be used to plan purifier movement in the virtual residential environment.

(b) A* considers both the cost already incurred and a heuristic estimate of the remaining cost. With an appropriate heuristic, it can efficiently find shortest paths in obstacle-filled grids.

(c) Since the apartment is represented as a two-dimensional grid, A* offers a clear implementation approach for obstacle-avoiding pathfinding.

6) Python Simulation — Virtual Environment

(a) Python will also be used to simulate the virtual residence and purifier operation without physical hardware.

(b) Simulation enables the team to reproduce air-quality changes, movement, and purification and to test the system repeatedly under different conditions.

(c) Python makes it straightforward to integrate the prediction model and pathfinding algorithm directly, while enabling controlled and repeatable experimental comparisons.

7) SQLite — Database

(a) SQLite is a lightweight relational database that does not require a separate database server. It will store demonstration accounts, simulation runs, air-quality readings and forecasts, and operation history.

(b) It stores data in a single file, is easy to configure and maintain, and integrates readily with Python.

(c) For a small-scale, locally operated prototype, SQLite is more practical than maintaining a dedicated database server.

8) WebSocket — Real-Time Communication

(a) WebSocket provides a persistent bidirectional connection between client and server. It will be used to stream simulation updates to the mobile app.

(b) Unlike a conventional request–response REST interaction, a WebSocket connection allows the server to push new data without a fresh client request each time.

(c) This is useful for frequently changing location, movement, and air-quality data; FastAPI also supports WebSocket integration.

9) Figma — UI/UX Design

(a) Figma will be used to design the mobile application interface and user experience before coding. It allows the team to visualize screen layouts and user flows and share designs.

(b) Its collaborative, web-based workflow helps team members review designs and agree on layouts, potentially reducing unnecessary implementation changes.

(c) The team may also explore an MCP integration to pass Figma design context to Claude Code and assist React Native UI implementation.

10) VS Code — Code Editor

(a) Visual Studio Code will be the primary code editor. It supports Python and TypeScript, with extensions for editing, debugging, and development-environment management.

(b) Using one editor for the mobile app and Python backend reduces the overhead of managing multiple development tools.

(c) Its Git integration also supports tracking changes and collaborating across the team.

11) Git + GitHub — Version Control and Collaboration

(a) Git is a distributed version-control system, while GitHub hosts repositories for online collaboration. They will be used to manage source code changes and integrate team contributions.

(b) Branches allow members to work independently, and pull requests support review before merging.

(c) The team will manage both source code and the LaTeX report in the repository to preserve a history of implementation and documentation changes.

12) Claude Code — AI-Assisted Development

(a) Claude Code is an AI coding assistant that supports code creation, editing, debugging, and codebase analysis. It will assist with development of the mobile application and Python backend.

(b) As the project integrates multiple frameworks and libraries, Claude Code may help the team understand code structure and streamline implementation and testing.

(c) A potential Figma MCP integration could provide design context for interface implementation. All AI-generated code will be reviewed and tested by team members for correctness and reliability.

4. Development Environment Specifications

Item	Specification
Development Hardware	Apple MacBook Air
Operating System	macOS Ventura 13.5 / Sequoia 15.1 / Tahoe 26.5.1
Processor	Apple M2–M3
Memory	8–16 GB RAM
Python	3.13.2
Node.js	v22.14.0
npm	10.9.2
Mobile Framework	React Native + Expo
Backend Framework	FastAPI
Machine Learning	scikit-learn
Database	SQLite
Development Tools	VS Code, Git, GitHub, Claude Code

The Python, Node.js, and npm versions were checked on one development computer. Other tool versions will be confirmed during environment setup.
Because team hardware and macOS versions differ, compatibility will be checked on the minimum configuration (M2, 8 GB RAM) and macOS Ventura 13.5.

5. Cost Estimation

Resource	Estimated cost
Existing Apple MacBook Air	No additional cost
Python and related libraries	Free
React Native + Expo	Free
FastAPI, SQLite	Free
VS Code, Git, GitHub Free	Free
Figma free plan	Free
Claude Code (assuming Claude Pro subscription)	Approx. US$20/month/account
AWS or Google Cloud	To be determined

The project will rely on existing equipment and mostly free, open-source software to minimize initial expenses. The Claude Code subscription amount is a planning estimate and may vary with the number of accounts, subscription period, and applicable terms.
Initial development and demonstrations will run locally. If cloud services are adopted later, costs will

be estimated based on the chosen server configuration and expected usage.

B. Software in Use

1. Existing Software and Technologies

Rather than creating new machine-learning or pathfinding algorithms from scratch, this project integrates existing technologies. A Random Forest regressor from scikit-learn will be used for prediction, while A* will be used for autonomous route planning. Python-based simulation will enable development and testing without physical hardware.

2. Main Algorithms and Software

1) Random Forest

Random Forest combines predictions from multiple decision trees. This project will use scikit-learn's RandomForestRegressor because it can learn nonlinear relationships among multiple input variables.

2) A Algorithm*

A* searches for a route by considering the cost already traveled and an estimated remaining cost. It is suitable for efficiently planning obstacle-avoiding routes in a grid-based environment.

3) Python Simulation

Python simulation will be used to construct a virtual environment and validate algorithms without physical hardware. It supports repeated experiments and comparisons under controlled conditions.

V. SPECIFICATIONS

A. Proposed Architecture and Execution Flow

1) System components:

The prototype is planned as a React Native application built with Expo, a Python backend built with FastAPI, and a lightweight SQLite database. The backend will host the virtual-environment engine, the trained Random Forest model, navigation logic, and operating controller in one application. No physical purifier or commercial cloud server is required for the first demonstration.

2) Data exchange:

The application will send user commands to FastAPI through REST endpoints. A WebSocket connection will deliver updates such as the virtual clock, purifier coordinates, room readings, forecast results, and operating status. A typical session will follow this flow: user login, dashboard loading, simulation start, environment update, prediction, destination decision, movement, local measurement, purification, and screen refresh.

3) State management:

The backend will maintain an explicit state for each run, including STOPPED, RUNNING, and PAUSED. It will separately track purifier activities such as IDLE, MOVING, MEASURING, and PURIFYING. These states will prevent contradictory actions, such as starting a second simulation loop while one is already running.

B. Login, Screen Transitions, and User Commands

1) Access process:

The initial loading screen will check for a stored session token. If a token is absent or invalid, the login screen will appear. The backend will compare the submitted password against a securely hashed value for a pre-created demonstration account in SQLite and will return a limited-duration session token on success. Passwords will not be stored as plaintext. A failed login will keep the user on the same screen and display a generic error.

2) Dashboard composition:

Following authentication, the main dashboard will load the predefined map and current run state. It will contain the floor plan at the center, a status area for virtual time and purifier operation, and controls for Start, Pause/Resume, Reset, and Automatic/Manual mode. Selecting a room will open a small information panel showing its latest measured value, measurement time, and 30-minute prediction.

3) Control behavior:

Start and Pause/Resume will change the simulation run state through backend requests. Reset will terminate the active run logic and restore the configured initial state before the next update is emitted. A manual destination command will be accepted only when Manual mode is enabled. When the mode changes, the backend will cancel a previously scheduled automatic destination to avoid competing commands.

4) Practical scope:

The sign-in function is included to describe the complete user flow, but the first version will not provide public registration, password reset, social login, multi-household management, or remote device pairing. These functions do not contribute to evaluating predictive movement and would increase development scope.

C. Virtual Apartment, Pollution Events, and Sensor Data

1) Apartment representation:

A fixed two-dimensional grid will represent walls, walkable areas, doors, and furniture. Each walkable cell will belong to a room or connecting corridor, and each room will have an identifier and a reachable entry point. The same coordinates will be shared by the simulator and the app so the displayed purifier position corresponds to the calculated route.

2) Air-quality engine:

The simulation clock will advance in one-minute steps, with room-level observations logged every 10 virtual minutes. The engine will modify each room's PM2.5 based on its previous concentration, probabilistically scheduled household events, natural decay, random background variation, and any active purification in that room. Values will be bounded at zero. The formulas and parameters are simplified experimental assumptions, not calibrated claims about an actual purifier.

3) Historical dataset and observation records:
An offline generator will create 60 days of synthetic history for three rooms, approximately 25,920 room-time values. To generate realistic forecasting inputs, separate simulated patrol runs will create observation histories that contain only the measurements a moving purifier could obtain. Hidden room states will remain available to the simulator for training labels and evaluation, but they will not be passed to the running decision controller.

4) Local sensing:
Once the purifier arrives, the simulator will allow a short virtual measurement interval before writing a new observation. Every observation will store the room, PM2.5 value, virtual timestamp, and run identifier. Other rooms will retain their last-seen readings. This distinction is essential because the proposed prototype has no separate sensor in every room.

D. Building and Using the Forecasting Model

1) Training target:
The selected model is RandomForestRegressor from scikit-learn. The target variable will be the simulated PM2.5 concentration for a given room 30 minutes after the input time, under a documented reference scenario without additional proactive purification. Training samples will be assembled from historical information that was available at the input time and labels from the simulator's subsequent ground-truth records.

2) Feature preparation and validation:
The planned features include room ID, hour of day, day of week, latest observed PM2.5, age of that reading, and historical room-level statistics calculated only from earlier observations. A missing-reading indicator will distinguish unvisited rooms. Days 1-42 will be used for training, days 43-51 for validation, and days 52-60 for final testing. Any input history that crosses a data split boundary will be excluded or constructed solely from earlier permitted records to prevent information leakage.

3) Model application:
After training, the model and its preprocessing configuration will be saved and loaded by the backend at startup. During Automatic mode, the controller will request a forecast for each room at fixed virtual intervals, initially every 10 minutes. Each result will include the room, predicted value, prediction target time, and generation time. A prediction failure will be recorded, and a fixed-patrol fallback will be used rather than silently inventing a forecast.

4) Interpretation:
The forecast indicates expected pollution if no new proactive intervention is introduced during the horizon. When the purifier moves early and reduces pollution, the resulting observed concentration may be lower than the original estimate. Such differences will be interpreted in context rather than automatically counted as forecasting errors caused by model failure.

E. Autonomous Navigation and Purification Control

1) Selecting a room:
At each decision interval, the controller will review predicted PM2.5 in all three rooms. A configurable experimental threshold will identify rooms that may require action. Candidate rooms will be ranked by estimated pollution risk with a penalty for travel time or route length. If no room presents meaningful predicted risk, the purifier may remain in place or follow the predefined inspection order.

2) Finding and following a route:
The grid map will be processed with the A* pathfinding algorithm, using the purifier's current cell as the start and the selected room's accessible cell as the goal. Adjacent traversable cells will form possible moves; walls and furniture will not. The result will be a list of grid coordinates. The movement engine will update the purifier one step at a time according to a fixed virtual speed and transmit each new location to the mobile app.

3) Arrival and cleaning:
When the purifier reaches the selected room, its status will change from MOVING to MEASURING. After a short measurement interval, the controller will read the local PM2.5 and decide whether to activate PURIFYING. While the purifier is operating, the room's PM2.5 will decrease gradually according to the simulation's removal parameter. The controller will periodically reconsider the plan using newly available observations and fresh forecasts.

4) Exceptions and stability:
If a destination does not exist, the calculated path is blocked, or the user changes to Manual mode, the current route will be discarded without teleporting the purifier. The system will also restrict routine destination switching to avoid rapid back-and-forth movement. A directly measured high-pollution condition may take precedence over the usual switching interval.

F. Mobile Visualization, Data Storage, and Operational History

1) Realtime view:

The floor-plan screen will draw the fixed map and render the purifier marker from backend coordinates. On each WebSocket update, the marker, target room, path, purifier state, and virtual time will be refreshed. A room panel will separately label Latest Measurement, Measured At, and Predicted PM2.5 in 30 Minutes. No remote room will be shown as having a fresh measured value unless the purifier has actually visited it.

2) Database records:
SQLite will maintain demonstration user accounts and sessions, simulation runs, virtual environment events, purifier observations, prediction records, movement decisions, and purification activities. Ground-truth pollution records will be stored separately for the experiment module. Each run will have an identifier so that resetting or replaying a scenario does not mix records from different attempts.

3) History and explanation:
A history screen will list major events in time order: predicted risk, selected destination, departure, arrival, measurement, and purification. Each automatic decision will store a short explanation such as "Higher predicted kitchen pollution; reachable in four virtual minutes." This history supports demonstration and later debugging rather than implying that the AI generates a natural-language explanation by itself.

4) Network failure:
On WebSocket disconnection, the app will display a disconnected status and preserve the last known values with their timestamps. After reconnection, it will request the current backend state before rendering new live information. The simulation will remain server-controlled, so reconnecting the app will not automatically start a second run.

G. Verification and Experimental Comparison

1) Functional checks:
The team will test login success and failure, session expiration, Start/Pause/Reset transitions, mode switching, unreachable destinations, sensor freshness labels, and recovery from disconnected mobile clients. Movement tests will confirm that no generated route crosses a blocked cell. Prediction tests will check that future data is excluded from model input.

2) Policy comparison:
To measure the value of prediction, the team will replay several held-out synthetic scenarios with two controllers: a fixed room-patrol baseline and the proposed prediction-based controller. Each pair of runs will use the same initial purifier position, pollution-event schedule, grid map, movement speed, purification rate, and sensor availability. This ensures that the main difference is the destination-selection policy.

3) Reported results and limits:
The final report will compare average PM2.5 across rooms, total time over a stated experimental threshold, and purifier travel distance. Model prediction error on unseen synthetic data will be reported separately. No improvement will be claimed until the simulation results have been measured. Because the environment and data are synthetic, outcomes will be described as simulation results rather than verified performance in real buildings.

REFERENCES

[1] LG Electronics, “LG PuriCare AI Objet Collection 360° Air Purifier M7,” model AS356NSMAM, official product information. https://www.lge.co.kr/product/air-purifier/as356nsmam

[2] LG Electronics, “LG PuriCare AI+ Air Purifier with AI Air-Quality Sensing,” LG Electronics Newsroom, Jan. 22, 2025. https://www.lge.co.kr/story/newsroom/229218

[3] LG Electronics, “LG Electronics Launches AI Air Purifier M7 with an Advanced Filter,” LG Electronics Newsroom, Jan. 30, 2026 (Korean). https://www.lge.co.kr/story/newsroom/235386

[4] ECOVACS ROBOTICS, “How do I use linked purification?”, ECOVACS Global Support, updated Aug. 30, 2022. https://help.ecovacs.com/global/support/faq-detail?id=465

[5] Python Software Foundation, Python Documentation. https://docs.python.org/3/

[6] scikit-learn Developers, scikit-learn Documentation. https://scikit-learn.org/

[7] Meta, React Native Documentation. https://reactnative.dev/

[8] Expo, Expo Documentation. https://docs.expo.dev/

[9] FastAPI, FastAPI Documentation. https://fastapi.tiangolo.com/

[10] SQLite, SQLite Documentation. https://www.sqlite.org/docs.html

[11] Anthropic, Claude Code Documentation. https://code.claude.com/docs/

[12] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis. for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, 1968.

[13] L. Breiman, “Random Forests,” Machine Learning, 2001.